

Computer Programming 1

Laboratory 06

(functions and procedures – part 1)

Example 1: Print uppercase

- Given a character in input, write a program that checks - with a function named *checkCharacter* - whether the character provided in input is a lowercase letter of the alphabet. If so, the program must print the uppercase character on the screen, using another function for the conversion (named *convertCharacter*).

Constraint: do not use arrays

E.g., 'a' => 'A'

Types of implementations:

- (1) By using functions of the ctype library (e.g., isalnum, isalpha, islower, tolower, etc.)
- (2) Without using functions in the ctype library (e.g., isalnum, isalpha, islower, tolower, etc.)

Example 2: Print uppercase v2

- Given a character in input, write a program that checks - with a function - whether the character provided in input is a lowercase letter of the alphabet. If so, the program must print the uppercase character on the screen, using a **procedure** for the conversion.

E.g., 'a' => 'A'

Constraint: do not use arrays

Types of implementations:

- (1) Without using functions in the ctype library (e.g., isalnum, isalpha, islower, tolower, etc.)
- (2) By using functions of the ctype library (e.g., isalnum, isalpha, islower, tolower, etc.)

Example 3: Revert number (a)

- Write a program that takes a number (of more than one digit) as input and prints the inverted number on the screen using a **procedure** named `printReverse()`, like in the following examples:

E.g.,

345 → 543

1234 → 4321

Constraint: do not use arrays

Example 3: Revert number (b)

Variant #2:

- Write a program that takes a number (of more than one digit) as input and prints the inverted number on the screen using a **function** named `printReverse()`, declared as below, like in the following examples

```
int printReverse(int number)
```

Constraint: do not use arrays

E.g.,

345 → 543

1234 → 4321

Example 4: Liar's Dice

- Write a program that:
 1. The program asks the user to guess how many dices turned out to be 1
 2. then it rolls 10 6-sided dices by using the function *rand()* of the library *cstdlib*
 3. If the number specified by the user corresponds to the number of 1 of the random dices, the user is the winner, and a message is printed.
 4. Otherwise, the number inserted by the user is wrong, a message is printed and the program execution ends

Implementation constraint: use a procedure called `create_dice_roll` for the step 2.

Constraint: do not use arrays

```
#include <cstdlib>
srand(time(NULL));
rand()
```

For additional details:

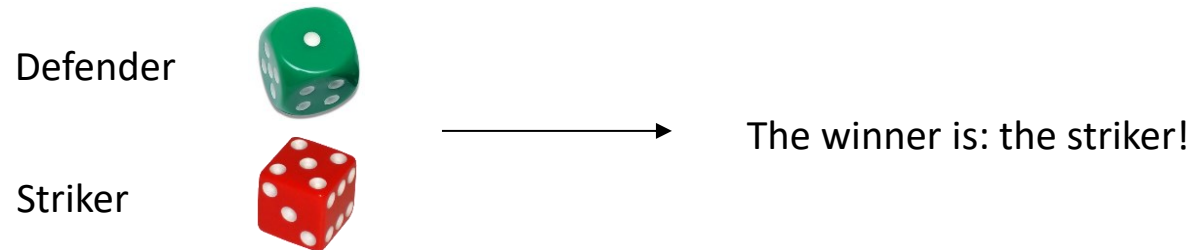
<http://www.cplusplus.com/reference/cstdlib/rand/>

Example 5: RisiKo! 1vs1 (a)

Write a program that simulates a 1-on-1 attack on RisiKo!.

1. Roll one 6-sided dice for the attacker and one dice for the defender.
2. Finally, declare the highest number as the winner.

Constraint: do not use arrays

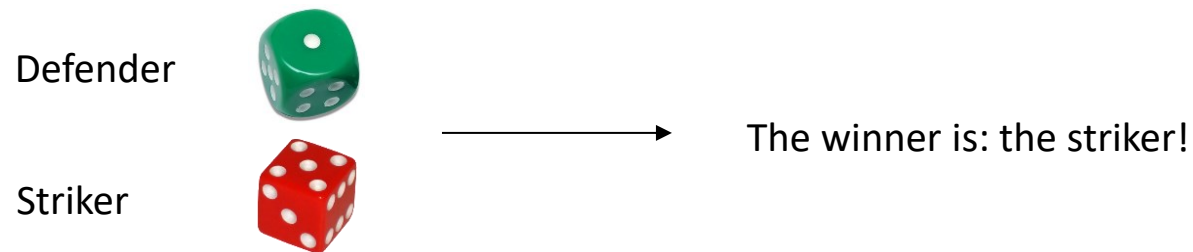


Implementation constraint: implement step 2 by means of one specific function named **compare** (i.e., additional function with respect to the *main*) that uses the parameter passing by values

Example 5: RisiKo! 1vs1 (b)

Write a program that simulates a 1-on-1 attack on RisiKo!.

1. Roll one 6-sided dice for the attacker and one dice for the defender.
2. Finally, declare the highest number as the winner.



Constraint: do not use arrays

Constraint: implement two functions (i.e., apart the *main*) that use the parameter passing by reference respectively for step 1 and step 2

```
void generate(int maxNum, int minNum, int& diceStiker, int & diceDefender);  
void compaire(int diceStiker, int diceDefender);
```

Example 6: RisiKo! 2vs2

Write a program that simulates a 2-on-2 attack on RisiKo!.

1. Roll 2 6-sided dices for the attacker and 2 dices for the defender.
2. Compare the highest die of the attacker against the highest of the defender, and the lowest of the attacker against the lowest of the defender.
3. Finally, declare the clashes won by the attacker and those won by the defender.

Constraint: do not use arrays

Implementation constraint: implement step 2 by means of one specific function named **compare** (i.e., additional function with respect to the *main*) that uses the parameter passing by values